

Q1. Mark the following statements as true or false.

1. Asynchronous programming is a form of programming in which the program executes every operation line by line before another operation can proceed. ()
2. Asynchronous operations let the program continue useful work while it waits for another operation to finish. ()
3. An uncompleted Future is already completed and has returned its data. ()
4. A synchronous function performs at least one asynchronous operation and can also perform synchronous operations. ()
5. You can use FutureBuilder to use or display data returned by a Future. ()
6. snapshot.error returns the latest data, and snapshot.data returns the newest error object. ()
7. Uncompleted describes a Future before it has produced a value or error. ()
8. A synchronous function is a function labeled with the async keyword. ()
9. FutureBuilder can represent waiting, error, and data through its AsyncSnapshot. ()
10. FutureBuilder should normally receive a stable Future rather than a newly created request on every build. ()
11. A Stream can deliver multiple values over time and then complete or report an error. ()
12. An async* function returns a Stream and can use yield to emit values over time. ()
13. await for consumes stream events in sequence until the stream is done or errors. ()
14. A single-subscription stream is not automatically safe for multiple simultaneous listeners. ()

Code fragment:

```
void main() {  
  print('Method 1');  
  print('Method 2');  
  print('Method 3');  
}
```

15. The following direct print fragment completes its direct statements before any delayed callback can print. ()

Code fragment:

```
void main() {  
  print('Method 1');  
  Future.delayed(const Duration(seconds: 2), () => print('Method 2'));  
  print('Method 3');  
}
```

16. The following delayed fragment prints Method 1 and Method 3 immediately, then Method 2 later. ()



Code fragment:

```
Future<String> fetchName() async {
  await Future.delayed(const Duration(seconds: 2));
  return 'Ruqaih Salman';
}

Future<void> main() async {
  print('Before request');
  final name = await fetchName();
  print('Name: $name');
  print('After request');
}
```

17. The following awaited fetch pauses the async function while the Future is pending, then resumes with the name. ()

Code fragment:

```
FutureBuilder<String>(  
  future: storedFuture,  
  builder: (context, snapshot) {  
    if (snapshot.hasError) {  
      return ElevatedButton(onPressed: retry, child: const Text('Retry'));  
    }  
    return const CircularProgressIndicator();  
  },  
)
```

18. In this FutureBuilder, the Retry branch represents a completed failure rather than a waiting state. ()

Code fragment:

```
FutureBuilder<String>(  
  future: storedFuture,  
  builder: (context, snapshot) {  
    if (snapshot.hasData) return Text(snapshot.requireData);  
    return const CircularProgressIndicator();  
  },  
)
```

19. In this FutureBuilder, the data branch represents a successful completed value, including an explicitly empty value when the design permits it. ()

Code fragment:

```
StreamBuilder<DateTime>(  
  stream: clockStream(),  
  builder: (context, snapshot) => Text('${snapshot.data}'),  
)
```

20. In this StreamBuilder, a new DateTime event can cause a rebuild without the caller manually requesting every frame. ()



Code fragment:

```
Stream<int> counterStream() async* {  
  for (var value = 1; value <= 5; value++) {  
    yield value;  
  }  
}
```

21. A counter stream can emit each value to the subscription and then expose a done state after the generator finishes. ()



Q2. Choose the correct answer from the following multiple choices

1. Which programming style allows work to continue while a single isolate waits for an event?

- A. Synchronous
- B. Asynchronous
- C. A and B
- D. None of the mentioned

2. Which Dart/Flutter APIs are used to implement asynchronous programming?

- A. Future
- B. async/await
- C. Future and async/await
- D. None of the mentioned

3. Which set contains common asynchronous operations?

- A. Fetching data over a network
- B. Writing to a database
- C. Reading data from a file
- D. All of the mentioned

4. Which states can a Future have?

- A. Uncompleted
- B. Completed with data
- C. Completed with an error
- D. All of the mentioned

5. Which parameters must a FutureBuilder normally receive?

- A. future
- B. builder function
- C. future and builder function
- D. None of the mentioned

6. Which keywords make asynchronous operations read like sequential code?

- A. async
- B. await
- C. async and await
- D. Futur

7. Which widget builds a widget when a Future resolves?

- A. FutureBuilder



- B. BuilderFuture
- C. Future
- D. None of the mentioned

8. What is the immutable representation of the latest interaction with an asynchronous computation?

- A. AsyncSnapshot
- B. data
- C. info
- D. None of the mentioned

9. Which pair completes an asynchronous expression and labels the containing function?

- A. async, await
- B. await, async
- C. await, Future
- D. None of the mentioned

10. A function returns Future<String>. What does that contract mean?

- A. It eventually completes with one String or an error
- B. It emits an unlimited number of Strings
- C. It returns a widget immediately
- D. It can never fail

11. What does await do inside an async function?

- A. Suspends that function until the Future completes
- B. Blocks every event in the application
- C. Creates a new isolate automatically
- D. Converts a Stream into a database

Code fragment:

```
Future.delayed(const Duration(seconds: 2), () => print('later'));  
print('next line');
```

12. The code calls Future.delayed but does not await or return it. What can happen?

- A. The surrounding function may continue before the delayed callback runs
- B. The delay becomes a synchronous sleep
- C. All later code is cancelled
- D. The Future becomes a Stream

13. Which code expresses “start this task and use its result before continuing”?

- A. final value = await loadValue();



- B. final value = print(loadValue());
- C. await Future<void> without a task
- D. yield loadValue() in a non-generator

14. A then callback returns a transformed value. What does the next stage receive?

- A. The transformed result
- B. The original widget tree only
- C. A route stack
- D. A database connection

15. Which error-handling structure is clearest for several awaited operations that share one recovery path?

- A. try/catch around the awaits
- B. A color change in build
- C. A new StreamBuilder with no stream
- D. Navigator.pop only

16. What is the purpose of finally in an async save flow?

- A. Restore cleanup or flags regardless of success or failure
- B. Retry forever
- C. Hide all errors from the user
- D. Convert a Future to a Stream

17. A request should stop waiting after five seconds. Which API expresses that deadline?

- A. Future.timeout
- B. Future.wait with one item
- C. StreamBuilder.initialData
- D. setState only

18. The timeout occurs, but the underlying operation cannot be forcibly cancelled by the Future API. What should the UI still do?

- A. Stop using the result and guard later State updates
- B. Assume the operation never runs
- C. Call dispose on the whole app
- D. Start an infinite retry

19. Task B finishes before task A in Future.wait([A, B]). What order is returned?

- A. [resultOfA, resultOfB]
- B. [resultOfB, resultOfA]
- C. Only B



D. Alphabetical order

20. The second request needs an id returned by the first. Which sequencing is appropriate?

- A. Await the first, then start the second with its id
- B. Start both with no dependency
- C. Use `initialData` for the id
- D. Use `cancelOnError` on a Future

21. A screen is popped while a Future is suspended. What risk exists when it completes?

- A. A later `setState` or context call may target an unmounted State
- B. The Future changes into a Stream
- C. The route automatically returns twice
- D. The request becomes a SQL query

22. Which snapshot state normally means a FutureBuilder Future is still running?

- A. `ConnectionState.waiting`
- B. `ConnectionState.doneWith data`
- C. `hasError` only
- D. `ConnectionState.active` for every Future

23. Why store a Future in State for a FutureBuilder profile screen?

- A. To avoid creating a new request on every build
- B. To make the Future persistent after uninstall
- C. To convert it into a Stream automatically
- D. To skip the loading state

24. Which abstraction is intended for multiple values over time?

- A. `Stream<T>`
- B. `Future<T>`
- C. `BuildContext`
- D. `RouteSettings`

25. What does `async*` return?

- A. A Stream
- B. A single String only
- C. A Widget
- D. A route name

26. What does `yield` do in an `async*` generator?

- A. Emit one event into the generated Stream



- B. Await a Future without suspension
- C. Pop a route
- D. Cancel every subscriber

27. Which StreamBuilder state can show that a finite stream has completed?

- A. ConnectionState.done
- B. ConnectionState.waiting only
- C. Future.timeout
- D. State.dispose



Q3. Predict the Async Code Output and Visible State

Read the code block before the question. Choose the exact console order or UI state that follows from the lecture model.

Code fragment:

```
void main() {  
  print('Method 1');  
  print('Method 2');  
  print('Method 3');  
}
```

1. Which console order follows?

- A. Method 1 → Method 2 → Method 3
- B. Method 1 → Method 3 → Method 2
- C. Method 2 only
- D. The order is undefined

Code fragment:

```
void main() {  
  print('Method 1');  
  Future.delayed(const Duration(seconds: 2), () => print('Method 2'));  
  print('Method 3');  
}
```

2. Which output appears before the delayed line?

- A. Method 1 → Method 2 → Method 3 immediately
- B. Method 1 → Method 3 immediately; Method 2 later
- C. Method 2 → Method 1 → Method 3
- D. Only Method 2

Code fragment:

```
Future<String> fetch() async {  
  await Future.delayed(const Duration(milliseconds: 10));  
  return 'Ruqaih Salman';  
}  
  
Future<void> main() async {  
  print('Before request');  
  final name = await fetch();  
  print('Name: $name');  
  print('After request');  
}
```

3. Which sequence is printed?

- A. Name first, then Before request
- B. Before request → After request → Name
- C. Before request → Name: Ruqaih Salman → After request



D. Only Before request

Code fragment:

```
Future<String> task(String label, int ms) async {
  await Future.delayed(Duration(milliseconds: ms));
  return label;
}
Future<void> main() async {
  final values = await Future.wait([task('A', 20), task('B', 5)]);
  print(values);
}
```

4. Which list is printed?

- A. [B, A]
- B. [A] only
- C. The list order is random
- D. [A, B]

Code fragment:

```
Future<void> main() async {
  try {
    await Future.delayed(const Duration(seconds: 2))
      .timeout(const Duration(milliseconds: 100));
  } on TimeoutException {
    print('Timed out');
  }
}
```

5. Which output follows?

- A. Timed out
- B. No output because the source is cancelled
- C. TimeoutException is printed automatically
- D. The two-second Future finishes first

Code fragment:

```
fetchName(fail: true)
  .then((name) => print('Name: $name'))
  .catchError((Object e) { print('Error: $e'); })
  .whenComplete(() => print('Finished'));
```

6. Which order follows when fetchName fails?

- A. Name line, then Finished
- B. Error line, then Finished
- C. Finished only
- D. Error line only; whenComplete is skipped



Code fragment:

```
Stream<int> values() async* {  
  for (var i = 1; i <= 3; i++) yield i;  
}  
Future<void> main() async {  
  await for (final v in values()) print(v);  
  print('Done');  
}
```

7. Which output follows?

- A. Done first, then 1, 2, 3
- B. Only 3 then Done
- C. 1, 2, 3, then Done
- D. The stream never closes

Code fragment:

```
void main() {  
  scheduleMicrotask(() => print('microtask'));  
  Future.delayed(Duration.zero, () => print('event'));  
  print('sync');  
}
```

8. Which order follows the event-loop model?

- A. microtask → sync → event
- B. sync → event → microtask
- C. event → microtask → sync
- D. sync → microtask → event

Code fragment:

```
Future<String> operation(int attempt) async {  
  if (attempt < 3) throw Exception('fail');  
  return 'ok';  
}  
// bounded retry calls attempts 1, 2, and 3 and returns on success
```

9. What happens?

- A. Attempts 1 and 2 fail; attempt 3 returns ok
- B. Attempt 1 returns ok
- C. All attempts run even after success
- D. The helper must retry forever



Code fragment:

```
FutureBuilder<String>(  
  future: storedFuture,  
  builder: (context, snapshot) {  
    if (snapshot.connectionState != ConnectionState.done) {  
      return const CircularProgressIndicator();  
    }  
    if (snapshot.hasError) return const Text('Error');  
    return Text(snapshot.requireData);  
  },  
)
```

10. Which state sequence is represented?

- A. ConnectionState.active is required for every Future
- B. Loading while incomplete, then Error or data when done
- C. It always displays data immediately
- D. It cannot render errors



Q4. Match Async Code with the Correct Console or Screen

Match each numbered code/state fragment with the correct labeled visual. Write the letters only; justify the match from the stated state.

Code 1:

```
void main() { print('Method 1'); print('Method 2'); print('Method 3'); }
```

Code 2:

```
void main() { print('Method 1'); Future.delayed(Duration.zero, () => print('Method 2')); print('Method 3'); }
```

Code 3:

```
Future<void> main() async { print('Before request'); final n = await Future.value('Ruqaih Salman');  
print('Name: $n'); print('After request'); }
```

Code 4:

```
Stream<int> s() async* { for (var i = 1; i <= 5; i++) yield i; } Future<void> main() async { await for (final i  
in s()) print(i); print('Stream closes'); }
```

A. Console: Method 1, Method 2, Method 3.	B. Console: Method 1, Method 3, then Method 2.
C. Console: Before request, Name: Ruqaih Salman, After request	D. Console: 1, 2, 3, 4, 5, then Stream closes.

Code 1: ____ Code 2: ____ Code 3: ____ Code 4: ____

Q5. Match FutureBuilder/StreamBuilder State with the Correct Screen

Match each numbered code/state fragment with the correct labeled visual. Write the letters only; justify the match from the stated state.

Code 1:

```
if (snapshot.connectionState != ConnectionState.done) return const CircularProgressIndicator();
```

Code 2:

```
if (snapshot.hasError) return ElevatedButton(onPressed: retry, child: const Text('Retry'));
```

Code 3:

```
return Text('Name: ${snapshot.requireData}');
```

Code 4:

```
final time = snapshot.requireData; return Text('${time.hour.toString().padLeft(2, '0')}:24:08');
```

A. Waiting screen with a progress indicator.	B. Error screen with a Retry action.
--	--------------------------------------



C. Completed data screen showing the fetched name.

D. StreamBuilder clock screen showing the latest time event.

Code 1: ____ Code 2: ____ Code 3: ____ Code 4: ____



Q6. Async Code and Screen-State Visual Analysis

The code/state description is part of every item. Choose one labeled visual and justify it; do not rely on remembering a screenshot from the lecture.

State/code description:

```
FutureBuilder<String>(future: storedFuture, builder: (context, snapshot) { if (snapshot.connectionState != ConnectionState.done) return const CircularProgressIndicator(); return Text(snapshot.requireData); })
```

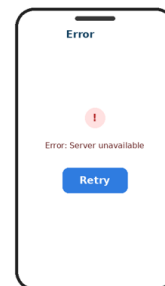
1. The Future is waiting before any value or error is available. Which labeled visual is correct?

A. Waiting screen with spinner	B. Error screen with Retry
C. Data screen with a name	D. Clock event screen

A.



B.



C.



D.



Choice: _____ Justification: _____



State/code description:

```
FutureBuilder<String>(future: storedFuture, builder: (context, snapshot) { if (snapshot.hasError) return  
ElevatedButton(onPressed: retry, child: const Text('Retry')); return const CircularProgressIndicator(); })
```

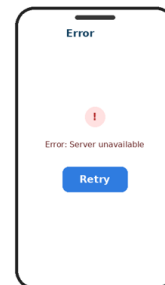
2. The Future has failed and the error branch is active. Which labeled visual is correct?

A. Waiting screen	B. Error screen with Retry
C. Data screen	D. Clock event screen

A.



B.



C.



D.



Choice: _____ Justification: _____

State/code description:

```
FutureBuilder<String>(future: storedFuture, builder: (context, snapshot) => Text('Name:  
${snapshot.requireData}'))
```

3. The Future completed with profile data. Which labeled visual is correct?

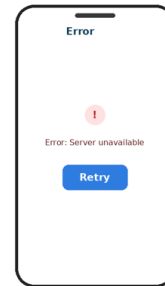
A. Waiting screen	B. Error screen
C. Data screen with the name	D. Clock event screen



A.



B.



C.



D.



Choice: _____ Justification: _____

State/code description:

```
StreamBuilder<DateTime>(stream: clockStream(), builder: (context, snapshot) { final time = snapshot.requireData; return Text('${time.hour.toString().padLeft(2, '0')}:24:08'); })
```

4. A clock StreamBuilder has received a new DateTime event. Which labeled visual represents the event-driven screen?

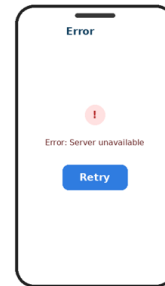
A. Waiting screen	B. Error screen
C. Profile data screen	D. Clock event screen



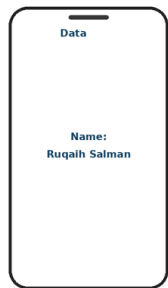
A.



B.



C.



D.



Choice: _____ Justification: _____

State/code description:

```
void main() { print('Method 1'); print('Method 2'); print('Method 3'); }
```

5. Direct synchronous print calls execute without a delayed callback. Which labeled console is correct?

A. Method 1, Method 2, Method 3	B. Method 1, Method 3, Method 2
C. Before request / Name / After request	D. 1, 2, 3, 4, 5, Stream closes



A

```
Console
Method 1
Method 2
Method 3
```

B

```
Console
Method 1
Method 3
Method 2
```

C

```
Console
Before request
Name: Ruqaih Salman
After request
```

D

```
Console
1
2
3
4
5
Stream closes
```

Choice: _____ Justification: _____

State/code description:

```
void main() { print('Method 1'); Future.delayed(Duration.zero, () => print('Method 2')); print('Method 3'); }
```

6. The delayed callback is scheduled after Method 1 and Method 3. Which labeled console is correct?

A. Method 1, Method 2, Method 3	B. Method 1, Method 3, Method 2
C. Before request / Name / After request	D. 1, 2, 3, 4, 5, Stream closes



A

```
Console
Method 1
Method 2
Method 3
```

B

```
Console
Method 1
Method 3
Method 2
```

C

```
Console
Before request
Name: Ruqaih Salman
After request
```

D

```
Console
1
2
3
4
5
Stream closes
```

Choice: _____ Justification: _____

State/code description:

```
Future<void> main() async { print('Before request'); final n = await Future.value('Ruqaih Salman');
print('Name: $n'); print('After request'); }
```

7. The awaited fetch prints before, then the name, then after. Which labeled console is correct?

A. Method 1, Method 2, Method 3	B. Method 1, Method 3, Method 2
C. Before request / Name / After request	D. 1, 2, 3, 4, 5, Stream closes



A

```

Console
Method 1
Method 2
Method 3

```

B

```

Console
Method 1
Method 3
Method 2

```

C

```

Console
Before request
Name: Ruqaih Salman
After request

```

D

```

Console
1
2
3
4
5
Stream closes

```

Choice: _____ Justification: _____

State/code description:

```

Stream<int> s() async* { for (var i = 1; i <= 5; i++) yield i; } Future<void> main() async { await for (final i in s()) print(i); print('Stream closes'); }

```

8. A counter Stream emits 1–5 and closes. Which labeled console is correct?

A. Method 1, Method 2, Method 3	B. Method 1, Method 3, Method 2
C. Before request / Name / After request	D. 1, 2, 3, 4, 5, Stream closes



A

```
Console
Method 1
Method 2
Method 3
```

B

```
Console
Method 1
Method 3
Method 2
```

C

```
Console
Before request
Name: Ruqaih Salman
After request
```

D

```
Console
1
2
3
4
5
Stream closes
```

Choice: _____ Justification: _____



Q7. Write Dart/Flutter Code for the Stated Async Contract

Equivalent null-safe implementations are acceptable when they preserve the stated Future/Stream, lifecycle, timeout, and visible-state requirements.

1. Write a delayed method-order example.

- Print Method 1 synchronously.
- Schedule Method 2 with Future.delayed.
- Print Method 3 synchronously; immediate order is Method 1 then Method 3.

Code response:

2. Write a Future<String> fetch and consume it with await.

- Wait about two seconds with Future.delayed.
- Return Ruqaih Salman.
- Print Before request, Name, and After request in the correct order.

Code response:

3. Write a timeout and bounded-retry helper.

- Use Future.timeout for a deadline.
- Handle TimeoutException distinctly.
- Use a finite attempt count and rethrow after the final failure.

Code response:

4. Write two independent tasks using Future.wait.

- Start A(2s) and B(1s).
- Return/print the results in input order A then B.
- Do not serialize independent work unnecessarily.

Code response:



5. Write a manual StatefulWidget loading/data/error pattern.

- Start `_load()` in `initState`.
- Check `mounted` after `await` before `setState`.
- Render spinner, error text, or the fetched value.

Code response:

6. Write a duplicate-action-safe save flow.

- Use `bool _saving` and return if already saving.
- Disable the button while saving.
- Use `try/finally` to restore the flag with a `mounted` check.

Code response:

7. Write a FutureBuilder profile screen with refresh/retry.

- Store the `Future` in `State` before `build`.
- Render `waiting`, `error + Retry`, and `data` branches.
- Replace the stored `Future` for an explicit `refresh`.

Code response:

8. Write a StreamBuilder clock screen.

- Create a `Stream<DateTime>` with `async*` and `yield`.
- Store/obtain the stream before `build`.
- Render the latest event and explain subscription ownership.

Code response:

End of Lab 8 — Asynchronous Programming | Student Edition — No solutions included

